



Algoritmos e Programação

Prof. Me. Fiorin
andre.fiorin@iffarroupilha.edu.br



Recapitulando

- Na última aula falamos sobre
 - Pseudocódigo
 - Tipos estruturados de dados
 - Operadores
 - Aritméticos
 - Lógicos
 - Relacionais
 - Estrutura básica de um algoritmo



Aula 04

Introdução à linguagem C



Introdução à linguagem C

- **Histórico**

- Criada na década de 70
- Dennis Ritchie
- Sistema Operacional UNIX (também criado por ele!)
- Considerada uma linguagem de programação genérica.





Introdução à linguagem C

- **Vantagens**

- Conjunto compacto de palavras reservadas.
- Conjunto compacto de tipos de dados.
- Permite o uso de ponteiros.
- Os parâmetros das funções podem ser passados tanto por referência quanto pelo endereço de memória.



Introdução à linguagem C

- Características gerais

- *Case sensitive*
- Comandos escritos em minúsculo
- 32 Palavras reservadas padrões

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while



Introdução à linguagem C

- **Estrutura básica**

- Um programa em C é composto por uma ou mais funções;
- A única função obrigatória é a **main ()**;
- Toda função deve ser precedida de “()”, o que indica que se trata de uma função.
- Os símbolos “{” e “}” indicam o início e o fim de um bloco de instruções ou de uma função, respectivamente.
- A palavra **void** indica que a função não retorna nenhum valor.



Introdução à linguagem C

- Estrutura básica – Exemplo

```
void main()    // Primeira função a ser executada
{
    /*Aqui ficaria as declarações locais e as instruções
    do programa */
}
```




Introdução à linguagem C

● Comentários

- Os comentários tornam o código-fonte de um programa mais legível, o que facilita futuras manutenções.
- Os comentários em C podem ser feitos de duas maneiras:
 - `//` comentário de linha
 - `/*comentário em bloco*/`

➤ Exemplos:

```
1 // este texto não será analisado pelo compilador
2
3 /* Tudo o que estiver no intervalo entre os asteriscos
4    não será analisado pelo compilador
5
6    int temp = 37;
7
8    mesmo que seja uma declaração válida
9 */
```



Tipos primitivos de dados



Linguagem C – tipos primitivos de dados

- **Estruturas primitivas – tipos de dados**

- **Inteiro:** conjunto de inteiros, representados pelos números naturais.
 - Suporta as operações (+, -, /, * e %);
- **Real:** conjunto dos reais ou fracionários.
 - Suporta as operações (+, -, / e *);
- **Lógicos (booleanos):** assumem apenas valores lógicos (0 e 1).
 - Suporta apenas operações lógicas (and, or e not);
- **Caractere:** representam valores literais, podendo ser uma letra, um número ou um símbolo qualquer.
 - Suporta operações de concatenação, *substring* e tamanho;



Linguagem C – tipos primitivos de dados

- Estruturas primitivas – faixa de valores

Nome	Tamanho em bits	Faixa de valores
char	8	
int	16	-32.768 a 32.767
float	32	10^{-38} a 10^{38}
long int	32	-2.147.483.648 a 2.147.483.647
double	64	10^{-308} a 10^{308}



Linguagem C – tipos primitivos de dados

- Estruturas primitivas – Modificadores

- Os modificadores alteram o significado de um tipo de dado básico
- É possível aplicar modificadores em dados dos tipos **int** e **char**.
 - **signed**
 - **unsigned**
 - **long**
 - **short**
- No caso do tipo **double**, é possível aplicar somente o modificador **long**.
- No tipo **float** não pode ser aplicado modificadores.



Linguagem C – tipos primitivos de dados

- **Modificadores de estruturas básicas**

- **Long:** É empregado aos tipos de dados `int` e `double` para indicar um tipo de dado maior. Também pode ser declarado em conjunto com o `float`, mas isto seria a mesma coisa de usar o tipo `double`.
- **Short:** Muito utilizado para indicar valores inteiros curtos (`short int`). Normalmente possui o mesmo tamanho do tipo `int`, mas são armazenados em um numero menor de bytes.



Linguagem C – tipos primitivos de dados

- **Modificadores de estruturas básicas**

- **Signed:** Este modificador é usado em dados do tipo caractere (**char**) para permitir o uso de sinal. Também pode ser usado com inteiros, porém isto seria redundante, pois sua declaração padrão já permite um número com sinal.
- **Unsigned:** Não permite o uso de sinais. Este modificador não é indicado para o tipo `int`, pois podem ocasionar algumas complicações caso seja atribuído em algum lugar do código um valor negativo.



Variáveis



Linguagem C - variáveis

- **Definição de variável**

- São posições da memória previamente identificadas para armazenar as informações a serem utilizadas pelo programa. Podem ser divididas em duas classes:
 - **Variáveis locais:** São declaradas dentro de funções. Só podem ser utilizadas pelas instruções que estão dentro de um bloco ({}) em que foram declaradas. Só existem no momento em que o bloco está sendo executado.
 - **Variáveis globais:** São declaradas fora de todas as funções do programa. Ao contrário das variáveis locais, as variáveis globais podem ser acessadas de qualquer parte do programa e persistem na memória até o término da execução do programa.



Linguagem C - variáveis

- **Nomenclatura de variáveis**

- Os nomes de variáveis, sejam globais ou locais, devem seguir as seguintes regras:
 - O nome da variável deve conter um ou mais caracteres;
 - O primeiro caractere **sempre** deve ser uma letra;
 - Os caracteres subsequentes podem ser letras, números ou o caractere “_” que geralmente é usado para indicar a separação entre duas palavras;
 - O nome da variável ou identificador não pode ser igual às palavras-chaves já apresentadas, nem ter o mesmo nome de funções determinadas pelo usuário ou iguais as funções localizadas na biblioteca C.



Linguagem C - variáveis

- Nomenclatura de variáveis

- Exemplos:

CORRETO	INCORRETO
soma1	1soma
soma	soma!
area_triangulo	area triangulo

- **ATENÇÃO:** A linguagem C é *case sensitive*, ou seja, as letras maiúsculas se diferem das minúsculas. Desta formas as variáveis soma, SOMA e Soma são distintas.



Linguagem C - variáveis

- **Declaração de variáveis**

- Sintaxe:

- `<tipo> Nome_variável;`

- Exemplo:

- `float nota1;`

- A atribuição de valores para as variáveis é feita utilizando o sinal de igual “=” e obedece a seguinte sintaxe:

- `Nome_variável = expressão;`



Comandos de entrada e saída



Linguagem C – Comandos de entrada e saída

- As operações de entrada e saída são realizadas por funções próprias.
- Existe em C uma biblioteca padrão que possui as funções básicas necessárias.

```
#include<stdio.h>
```



Linguagem C – Comandos de entrada e saída

- **Comando de saída de dados**

- É o comando utilizado para exibir textos e valores de variáveis na tela.

- **Sintaxe**

- `printf ("<texto para impressão>", <expressão1>, <expressão2>, ...);`

- **onde:**

- **<texto para impressão>:** é o texto que será impresso na tela. Pode ser composto de caracteres que serão exibidos na tela e de caracteres especiais cuja função é indicar:
 - a) os locais e tipos de expressões que serão mostradas; ou
 - b) ações que serão executadas como pular uma linha ou emitir um bip no alto-falante.
 - **<expressão>:** argumento é opcional. Indica um valor que será impresso dentro do <texto para impressão>. Pode ser uma variável, constante, expressão aritmética ou lógica ou ainda uma chamada a outra função.



Linguagem C – Comandos de entrada e saída

- Comando de saída de dados

- Exemplos

- `printf ("Este texto será mostrado na tela.");`
 - `printf ("O valor da variável idade é %d.", idade);`
 - `printf ("Depois deste texto serão puladas duas linhas.\n\n");`



Linguagem C – Comandos de entrada e saída

- **Comando de saída de dados**

- Para imprimir o conteúdo de uma variável dentro do texto é necessário inserir neste mesmo texto no local de impressão um % seguido do tipo de dado a ser mostrado.
- As principais opções para impressão de variáveis são:

Código	Tipo de impressão
%d	Número na base decimal
%f	Número real (float)
%lf	Número real (double)
%c	Caractere
%s	String
%%	Sinal de porcentagem



Linguagem C – Comandos de entrada e saída

- **Comando de saída de dados**

- Também é possível especificar caracteres especiais que representam determinadas ações.

- Exemplos:

Código	Tipo de impressão
<code>\n</code>	Pula uma linha (nova linha)
<code>\t</code>	Tabulação
<code>\b</code>	Backspace
<code>\a</code>	Emite um beep no auto falante
<code>\"</code>	Imprime aspas (")
<code>\\</code>	Imprime contra-barra (\)



Linguagem C – Comandos de entrada e saída

- **Comando de entrada de dados**

- É utilizado para fazer a leitura dos dados informados pelo teclado.

- **Sintaxe**

- `scanf("<string de leitura>", &<variável 1>, &<variável 2>, ...);`

- **onde:**

- <string para leitura> é composta pelos tipos das variáveis para leitura, por caracteres que devem ser digitados junto com as variáveis e com formatação especial (entre colchetes);
 - &<variável> endereço da variável que receberá o valor que for digitado no teclado. Os valores digitados devem ser separados por um espaço em branco, uma tabulação ou pelo <enter>.



Linguagem C – Comandos de entrada e saída

- Comando de entrada de dados

- Exemplos

- `scanf("%d", &numero);`
 - Este comando aguarda a digitação de um valor numérico do teclado. O valor digitado é armazenado na variável `numero`.
 - `scanf("%c", &letra);`
 - Este comando aguarda a digitação de um caractere do teclado. O caractere digitado é armazenado na variável `letra`.
 - `scanf("%d %d", &num1, &num2);`
 - Este comando aguarda a digitação de dois valores numéricos do teclado. O primeiro valor digitado é armazenado em `num1` e o segundo em `num2`. Estes valores podem ser separados por espaço em branco, tabulação ou enter e finalizados por um enter.
 - `scanf("%d,%d", &num1, &num2);`
 - Este comando aguarda a digitação de dois valores numéricos do teclado. O primeiro valor digitado é armazenado em `num1` e o segundo em `num2`. A diferença com o anterior é a vírgula colocada entre os dois `%d`. Na execução do programa esta vírgula deve ser digitada entre os dois números.



Operadores aritméticos



Linguagem C: operadores aritméticos

- Operadores aritméticos

- São responsáveis pelas operações matemáticas realizadas no programa.

Nome	Operador	Exemplo
Atribuição	=	<code>x = 3;</code>
Adição	+	<code>x = 3 + 2;</code>
Subtração	-	<code>x = 3 - 2;</code>
Multiplicação	*	<code>x = 3 * 2;</code>
Divisão	/	<code>x = 3 / 2;</code>
Resto da divisão	%	<code>x = 3 % 2</code>



Linguagem C: operadores aritméticos

- Operadores aritméticos

- **Importante:** todas as operações são feitas na precisão dos operandos!
 - `x = 5 / 2;` resulta no valor 2, pois a operação de divisão é feita na precisão inteira, já que os dois operandos (2 e 5) são constantes inteiras.
 - A divisão de inteiros “trunca” (*trunc*) a parte fracionária, pois o valor resultante sempre será do mesmo tipo da expressão.
 - `x = 5.0 / 2.0;`
`x = ???`



Linguagem C: operadores aritméticos

- Operadores aritméticos

- O operador % produz o resto da divisão do primeiro operando pelo segundo.
- Só é aplicado em operandos do tipo **inteiro**.
- É utilizado para descobrir se um valor de uma determinada variável é par ou ímpar.
- Um número é par se $(x \% 2)$ for igual a zero.



Linguagem C: operadores aritméticos

- Operadores aritméticos

- A linguagem C permite utilizar operadores de atribuição compostos.

- Exemplo: `x = x + 2;`

- A variável `x` também aparece dos dois lados do sinal de atribuição (`=`).

- Este comando pode ser escrito de forma mais compacta, utilizando o operador de atribuição composto.

- Exemplos:

`x += 2;` `//` mesmo que `x = x + 2;`

`x *= (3 + 2);` `//` mesmo que `x = x * (3 + 2);`

`x %= 2;` `//` mesmo que `x = x % 2;`



Linguagem C: operadores aritméticos

- Operador de incremento e decremento

- ++

- --

- São usados para incrementar/decrementar em uma unidade o valor de uma variável. Exemplo:

- `x = 45;`

- `x++;`

`// x = 46`



Linguagem C: operadores aritméticos

- **Operador de incremento e decremento**

- Os operadores de incremento e decremento podem ser usados prefixados (antes da variável) ou pós-fixados (depois da variável).
- Exemplo:
`x++;`
`++x;`
- Em ambos os casos a variável `x` é incrementada, porém a expressão `n = x++` incrementa após o seu valor ser usado.
- Exemplo (tendo `x = 5`):
`n = x++;` // `n` recebe o valor 5; `x` é incrementado para 6
`n = ++x;` // `x` é incrementado para 6; `n` recebe o valor 6



Linguagem C: operadores aritméticos

- Operador de incremento e decremento

- Os operadores de incremento e decremento podem ser aplicados somente em variáveis.

`x = (i + 1)++;` // **INCORRETO!**

- Em síntese:

`a = a + 1;`

`a += 1;`

`a++;`

`++a;`

} Expressões equivalentes



Operadores relacionais



Linguagem C: operadores relacionais

- Operadores relacionais

- São aqueles que estabelecem uma relação entre dois operandos ou expressões.

- < // menor que
- > // maior que
- <= // menor ou igual que
- >= // maior ou igual que
- == // igual a
- != // diferente de

- Serve para comparar dois valores;



Linguagem C: operadores relacionais

- Operadores relacionais

- O resultado produzido por um operador relacional é 0 (zero) ou 1.
- O valor 0 (zero) é interpretado como **falso**.
- Qualquer valor diferente de zero é considerado **verdadeiro**.
- Assim, se o resultado de uma comparação for falso, produz-se o valor zero, caso contrário, produz-se o valor 1.



Operadores lógicos



Linguagem C: operadores lógicos

- Operadores lógicos

- Combinação de expressões booleanas.

- `&&` // operador binário E (AND)
- `||` // operador binário OU (OR)
- `!` // operador binário de NEGAÇÃO (NOT)

- Exemplo:

```
int a, b;  
int c = 23;  
int d = c + 4;  
a = (c < 20) || (d > c); // verdadeiro  
b = (c < 20) && (d > c); // falso
```



Linguagem C

- Exemplo

```
1 #include <stdio.h>
2
3 void main (void) {
4     float notal, nota2;
5     float media;
6
7     printf ("\n Informe a primeira nota: ");
8     scanf ("%f", &notal);
9     printf ("\n Informe a segunda nota: ");
10    scanf ("%f", &nota2);
11
12    media = (notal + nota2) / 2;
13
14    printf (" A media do aluno eh %f", media);
15    getch ();
16 }
```



Na próxima aula...

- Estruturas de decisão e de repetição em C



Referências

- SEBESTA, R. W. **Conceitos de Linguagem de Programação**. 4 ed. Bookman Companhia Ed. 2000.
- KERNIGHAM, Brian W.; RITCHIE, Dennis M. **C: A Linguagem de Programação**. Rio de Janeiro: Campus, 2002.
- DEITEL, H. M.; DEITEL, P. J. **Como Programar em C**. Rio de Janeiro: LTC - Livros Técnicos e Científicos, 1999.